

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	
Academic Year: 2025-2026			
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr. J. Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S. Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch. Rajitha	
		Mr. M Prakash	
		Mr. B. Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
NS_2 (Mounika)			
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week 6 - Wednesday	Time(s)	
Duration	2 Hours	Applicable to Batches	
NAME: VADDEM CHANDANA HALL TICKET NO: 2403A51186 BATCH: 09 Assignment Number: 11.3 (Present assignment number) / 24 (Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 11 – Data Structures with AI: Implementing Fundamental Structures Lab Objectives		Week 5 - Monday

	• To implement fundamental data structures with the assistance of AI tools. • To understand how AI suggests different implementations and optimizations. • To analyze the readability, correctness, and performance of AI-generated code. • To reinforce problem-solving skills using AI-powered coding assistance. Learning Outcomes After completing this lab, students will be able to: 1. Implement stack, queue, and linked list using Python with AI support. 2. Use AI tools to optimize and refactor basic data structure operations. 3. Compare multiple AI-suggested implementations for the same structure. 4. Apply AI assistance to generate test cases for verifying data structure behavior. 5. Demonstrate understanding of trade-offs in AI-generated solutions. Task Description #1 – Stack class implementation Task: Ask AI to implement a stack class with push(), pop(), peek() and is_empty() methods Code and output:	
--	--	--

Commands + Code + Text ▶ Run all ▼

```
[2]
✓ 0s class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        """Adds an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Removes and returns the item from the top of the stack."""
        if not self.is_empty():
            return self.items.pop()
        else:
            return "Stack is empty"

    def peek(self):
        """Returns the item at the top of the stack without removing it."""
        if not self.is_empty():
            return self.items[-1]
        else:
            return "Stack is empty"

    def is_empty(self):
        """Checks if the stack is empty."""
        return len(self.items) == 0

        return self.items[-1]
    else:
        return "Stack is empty"

def is_empty(self):
    """Checks if the stack is empty."""
    return len(self.items) == 0

# Example usage:
my_stack = Stack()
my_stack.push(10)
my_stack.push(20)
print(my_stack.peek())
print(my_stack.pop())
print(my_stack.is_empty())
```

```
20
20
False
```

Task Description #2 – Queue Implementation

Task: Use AI to generate a Queue class with enqueue(), dequeue(), and is_empty().

Code and output:

```
[5] class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        """Adds an item to the rear of the queue."""
        self.items.insert(0, item) # Add to the beginning of the list

    def dequeue(self):
        """Removes and returns the item from the front of the queue."""
        if not self.is_empty():
            return self.items.pop() # Remove from the end of the list
        else:
            return "Queue is empty"

    def is_empty(self):
        """Checks if the queue is empty."""
        return len(self.items) == 0

# Example usage:
my_queue = Queue()
my_queue.enqueue(10)
my_queue.enqueue(20)
print(my_queue.dequeue())
print(my_queue.is_empty())
```

10
False

Task Description #3 – Linked List Implementation

Task: Ask AI to create a singly linked list with `insert_at_end()`, `insert_at_beginning()`, and `display()`.

Code and output:

```
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        """Inserts a new node with the given data at the end of the linked list."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def insert_at_beginning(self, data):
        """Inserts a new node with the given data at the beginning of the linked list."""
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```

```

        new_node.next = self.head
        self.head = new_node

    def display(self):
        """Prints the data of each node in the linked list."""
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

# Example usage:
my_list = SinglyLinkedList()
my_list.insert_at_end(10)
my_list.insert_at_beginning(5)
my_list.insert_at_end(20)
my_list.display()

```

5 -> 10 -> 20 -> None

Task Description #4 – Binary Search Tree (BST)

Task: Ask AI to generate a simple BST with insert() and inorder_traversal().

Code and output:

```

class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key

class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        """Inserts a new node with the given key into the BST."""
        if self.root is None:
            self.root = Node(key)
        else:
            self._insert_recursive(self.root, key)

    def _insert_recursive(self, root, key):
        if key < root.val:
            if root.left is None:
                root.left = Node(key)
            else:
                self._insert_recursive(root.left, key)
        else:
            if root.right is None:
                root.right = Node(key)

```



```
        else:
            self._insert_recursive(root.right, key)

    def inorder_traversal(self):
        """Performs an inorder traversal of the BST and prints the result"""
        result = []
        self._inorder_recursive(self.root, result)
        print("Inorder Traversal:", result)

    def _inorder_recursive(self, root, result):
        if root:
            self._inorder_recursive(root.left, result)
            result.append(root.val)
            self._inorder_recursive(root.right, result)

# Example usage:
my_bst = BST()
my_bst.insert(50)
my_bst.insert(30)
my_bst.insert(20)
my_bst.insert(40)
my_bst.insert(70)
my_bst.insert(60)
my_bst.insert(80)
my_bst.inorder_traversal()
```



Inorder Traversal: [20, 30, 40, 50, 60, 70, 80]